

IES Top and Core First release candidates

Briefing pack v1.0 (Oct 2025)

Introduction

This document provides a briefing on the two first release candidates for IES-Top and IES-Core.

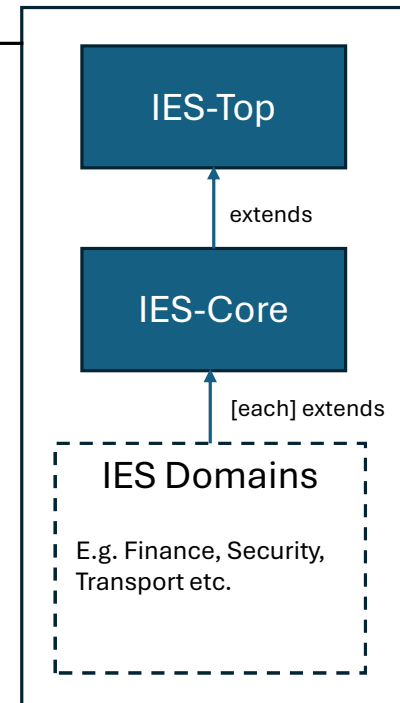
1. **IES-Top** – an RDF top-level ontology which is based on the BORO™ Foundational Ontology¹ and grounded in Extensional 4-Dimensionalism. It is also grounded in Pluralities², the Pluriverse and Constructionalism³ (see the Core Constructional Ontology⁴).
2. **IES-Core** – an RDF mid-level ontology built upon IES-Top providing a common foundational layer of patterns and concepts useful for expressing information in all manner of domains. Patterns such as Activities, Replaceable Parts, Characteristics and Representations. This also includes lower-level concepts like Persons, Locations etc. Core replaces the top layers of IES4. Domain specific ontologies are expected to be built upon IES-Core.

These are not finished pieces of work - IES-Top is mature, but some development is still required – mainly on IES-Core. These release candidates (RC1) are being provided to give the community the opportunity to review and comment. Feedback will be taken into two subsequent release candidates (RC2 and RC3) before the final publication of these new ontologies in spring/summer 2026.

Details of the work that still needs to be done are provided on the next slide.

NOTE: for the ease of referring to the whole stack of new IES ontologies, we will use the term IES-Next in this document.

“IES-Next”



1. de Cesare, S., & Partridge, C. (2016). BORO as a Foundation to Enterprise Ontology. *Journal of Information Systems*, 30(2 (Summer 2016)), 83–112.

2. Florio, S., & Linnebo, Ø. (2021). *The many and the one: A philosophical study of plural logic* (First edition). Oxford University Press.

3. Partridge, C., deCesare, S., Mitchell, A., Gailly, F., & Khan, M. (2017). *Developing an Ontological Sandbox: Investigating Multi-Level Modelling's Possible Metaphysical Structures*. (MODELS 2017), 2019, 226–234.

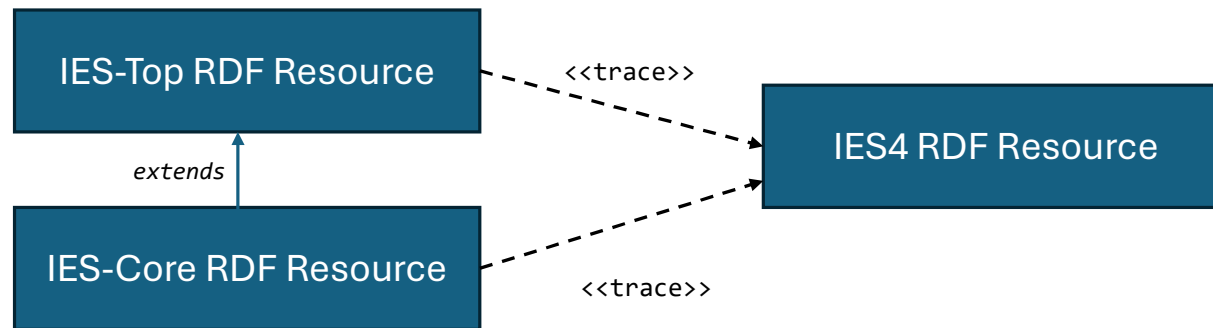
4. Florio, S., & Linnebo, Ø. (2022). *Core Constructional Ontology: The Foundation for the Top-Level Ontology of the Information Management Framework*.

Work still to be done

1. Develop IES-Core class and property definitions (with support from the community). The IES-Core RC1 definitions are drafts and many of them carry-over from IES4. We have the aspiration of creating definitions based on spatial-temporal extents.
2. Update the Location pattern in light of work being done in the community regarding the integration of IES with GeoSparql.
3. Update Measurements pattern in light of new analysis conducted in this area by members of the community.
4. Test and validate Entitlement/Rights pattern.
5. Test and validate Accounts pattern.
6. Develop a general Registration pattern (for use cases such as registeredCountry (for ships and companies) and registeredPort (for ships).
7. Redeveloping NamingScheme following change to how IES is extended. See slide 22 for information about said change.

Migration from IES4 to IES-Next

The development of the new major version of IES has included traceability between resources in the new stack to those in the current major version (at time of writing), IES4. As part of the final release of IES-Next, a migration tool will be provided to the community.



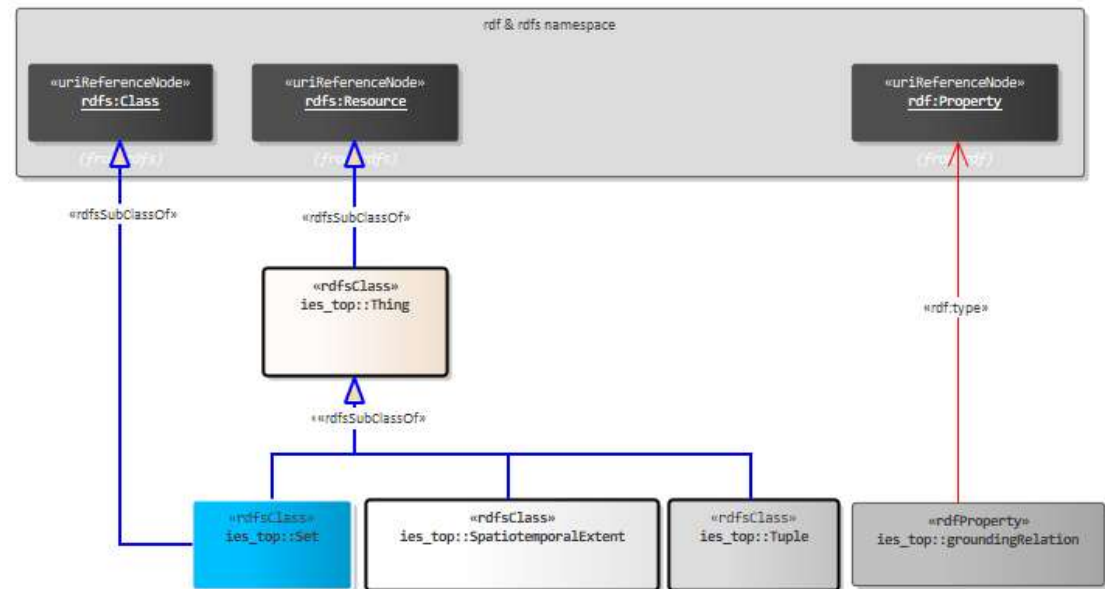
Design decisions

As mentioned in the introduction, the new IES stack will be grounded in topics such as Extensional 4-Dimensionalism and Pluralities. Translating these into a physical implementation (like RDF) inevitably requires making choices—whether due to limitations of the chosen implementation technology or for user experience reasons. Throughout this document, we call out design decisions made in this regard in purple call-out boxes.

Introducing IES-Top

The top of Top

- **Thing** – Is either a spatiotemporal extent, set, tuple or grounding relation.
- **Element** - A Thing that is a part of the pluriverse – and so has a 4D extent.
- **Set** – A Thing that is an unordered collection of other Things, sometimes referred to as a class or kind.
- **Tuple** - A Thing that is an (ordered) sequence of Things. Each position in the sequence is pointed to by a tuplePlace.
- **groundingRelation** – A thing which is one of the four basic relationships between two Things.



DESIGN DECISIONS:

1. The equivalent top level of the BORO™ Foundational Ontology and the Core Constructional Ontology, introduce higher-order relations that go beyond those available in RDF/S. At this level we are not talking about sub and super sets/classes but sub and super pluralities. However, in the implementation we have avoided adding such relations and instead stuck with using `subClassOf` and `subPropertyOf` which are ubiquitous among the RDF community and its tools.
2. In IES4, `Class` was used instead of `Set`. The use of the word `Class` encouraged users to think in an Object Orientated Programming way which led to confusion. In IES-Next we have elected to use the word `Set` to encourage users to think in a set-theoretic way.
3. The equivalent item to `groundingRelation` in the BORO™ is a sub-plurality just like a `Tuple` and `Set`. Because all grounding relations in the ToLO have one source and one target, we have realized `groundingRelation` in RDF, for simplicity, as a `rdf:Property`.

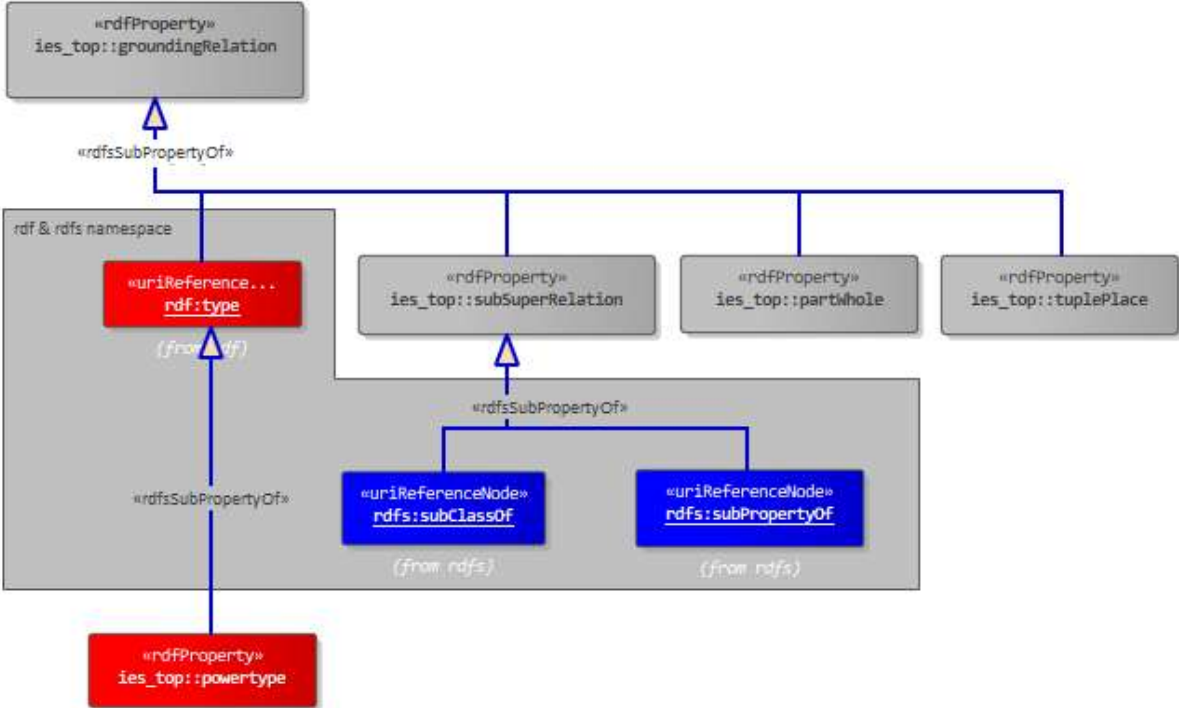
Grounding Relations

Grounding relations are derived from the four basic constructors outlined in the Core Constructional Ontology (CCO). They are:

1. type (Element-Set)
2. subSuperRelation (Subset-Superset)
3. partWhole (Part-Whole)
4. tuplePlace (Tuple Place)

Note: regarding the above, the first name is that used in IES-Top, the name in brackets is used in CCO.

Note: In the CCO, the term ‘element’ is used for the role in the Element-Set relationship. In IES we refer to this as ‘member’ i.e. Member-Set.

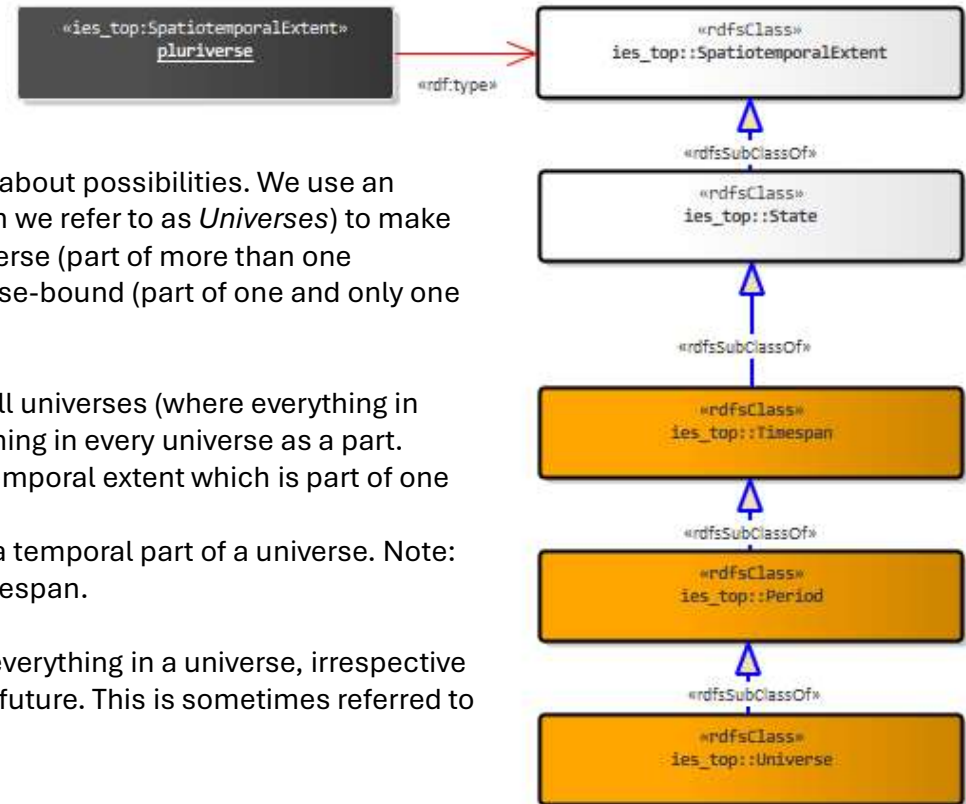


Note: As mentioned on the previous slide, the concept of pluralities is found at a higher-order than the concepts in RDF/S. As a result, we have had to define RDF/S resources in the context of our top-level. Here you will see that `rdf:type`, `rdfs:subClassOf` and `rdfs:subPropertyOf` are themselves `ies_top:groundingRelations`.

The Pluriverse

In IES, we commit to the pluriverse, which provides the grounding for talk about possibilities. We use an approach based on David Lewis' interpretation of *Possible Worlds*⁵ (which we refer to as *Universes*) to make clear distinctions between spatiotemporal extents that can be trans-universe (part of more than one universe) and the ones we do most of our work with, those that are universe-bound (part of one and only one universe).

- **pluriverse** - An instance of SpatiotemporalExtent which is the sum of all universes (where everything in each universe is part of that universe). Put another way, this has everything in every universe as a part.
- **State** – a spatiotemporal extent that is universe-bound i.e. a spatiotemporal extent which is part of one and only one universe.
- **Timespan** – a state (i.e. universe-bound spatiotemporal extent) that is a temporal part of a universe. Note: a universe is an improper temporal part of itself – and so a maximal timespan.
- **Period** – a connected and uninterrupted timespan.
- **Universe** – a maximal spatiotemporal connected extent that includes everything in a universe, irrespective of any indexing, such as whether it is present now, in the past, or in the future. This is sometimes referred to as a world.



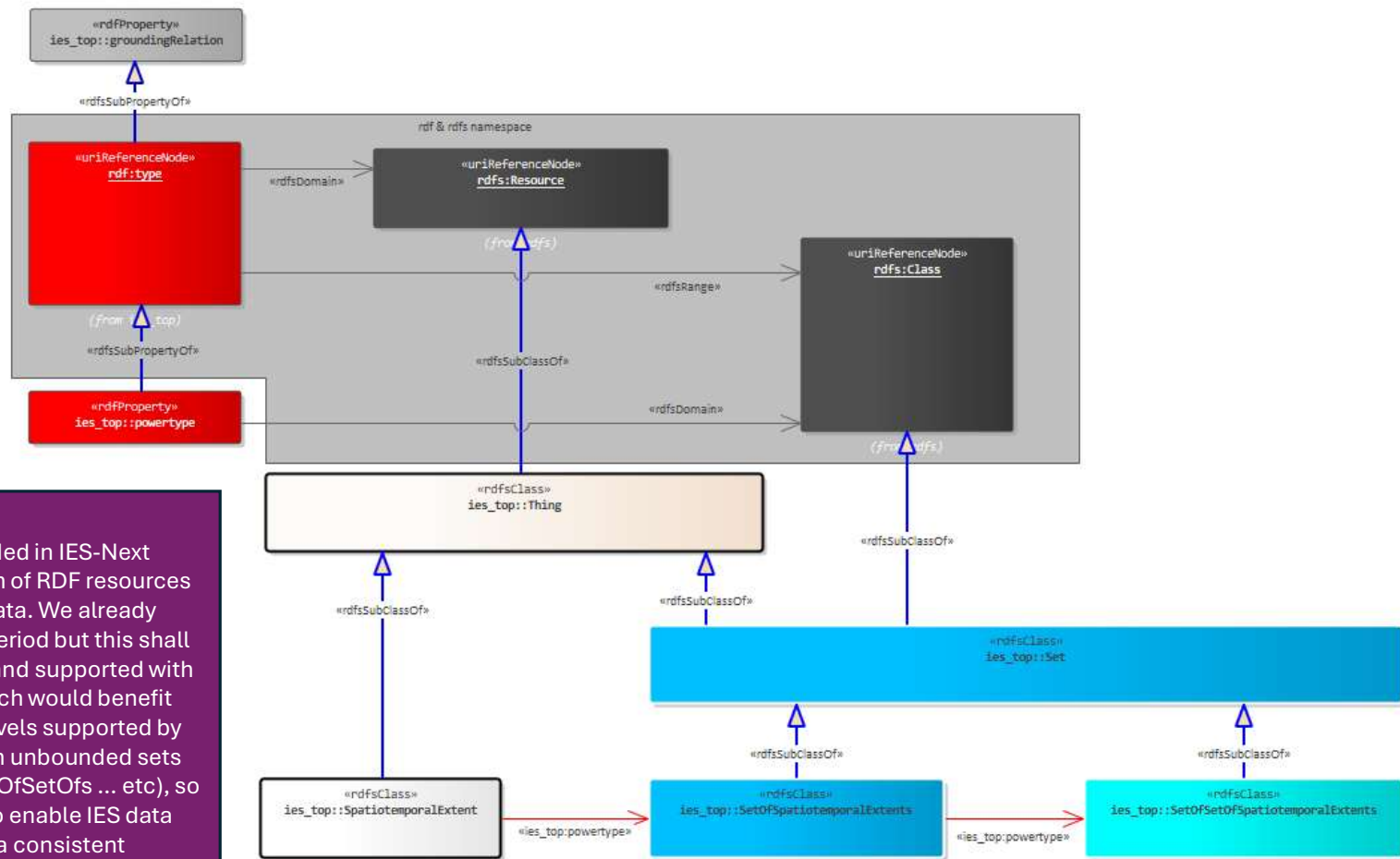
DESIGN DECISIONS:

4. A balance has been struck across the development of IES-Top in regard to the names of RDF resources. The balance being between using names that are familiar and easy for users to understand and those that are explicit in what they refer to from a 4D and Possible Worlds perspective. SpatiotemporalExtent and State being two examples of this. What was called Element in IES4 has been redefined as a Thing that could either be universe-bound or trans-universe and renamed to be what it always was – a SpatiotemporalExtent. The sort of spatiotemporal extents users will deal with 99% of time will be universe-bound extents. In order to avoid having a long and probably perplexing name (like UniverseBoundSpatiotemporalExtent) which would be used ubiquitously within instance data, we have opted to carry over the name State from IES4 but bound this SpatiotemporalExtent to a single universe.

5. D. K. Lewis, On the plurality of worlds. Malden, Mass: Blackwell Publishers, 1986.

Instances and types

To realise the need to be able to put elements into sets we use the ubiquitously used `rdf:type`. This is the same approach as in IES4. `powertype` also carries over from IES4.



DESIGN DECISIONS:

5. A new URI minting regime will be included in IES-Next which will support the consistent creation of RDF resources many users typically create within their data. We already have such a pattern for IES4's ParticularPeriod but this shall be extended to other areas of the model and supported with SHACL validation artefacts. One area which would benefit from this is the unbounded, infinite set levels supported by IES. It is not feasible to issue a model with unbounded sets (e.g. SetOfs, SetOfSetOfs, SetOfSetOfSetOfSetOfs ... etc), so a URI minting pattern shall be specified to enable IES data creators to create the levels they need in a consistent fashion.

Parts and wholes

For the mereological relations, we make clear distinctions between what spatiotemporal extents are universe-bound and which are not. An additional distinction is made here for mereologies between spatiotemporal extents in the same universe (universe-mates).

- **partWhole** – a grounding relation placing one spatiotemporal extent as part of another (the whole).
- **isPartOf** – a partWhole relation between two states, where both states are bound to the same universe (universe-mates).
- **isTemporalPartOf** – an isPartOf that asserts the spatial extent of the (whole) state is co-extensive with the spatial extent of the (part) state for a particular period of time.
- **isAStartOf** – an isTemporalPartOf that places a state as one (but not always the only) temporal part at the beginning of another.
- **isAFinishOf** – an isTemporalPartOf that places a state as one (but not always the only) temporal part at the conclusion of another.

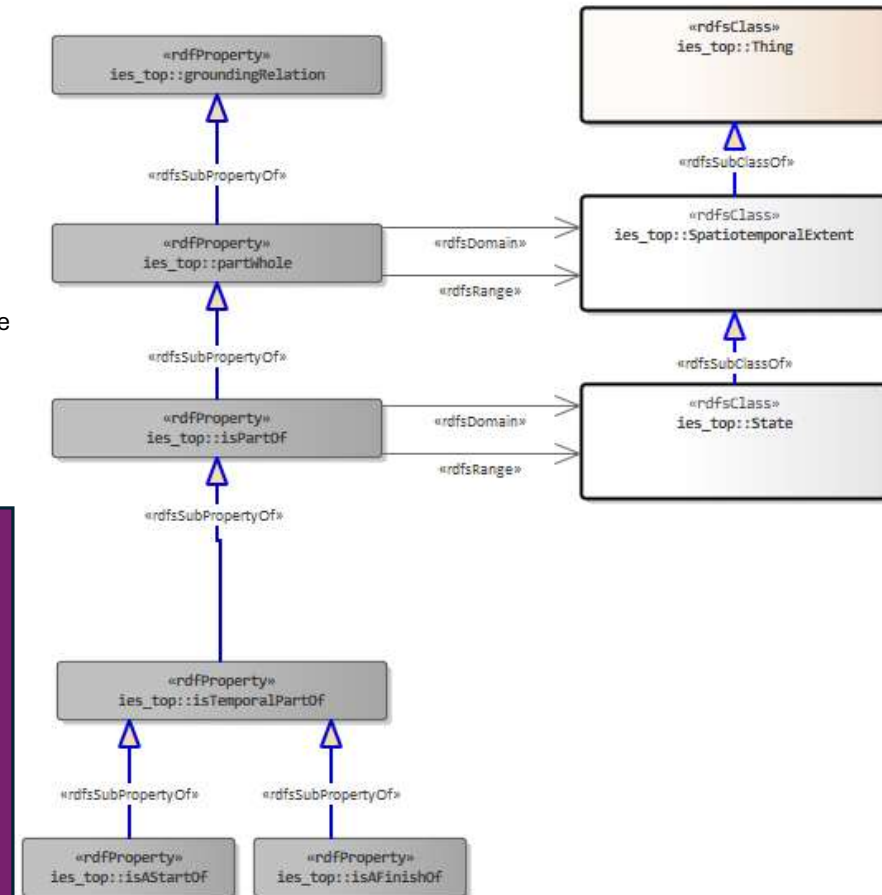
DESIGN DECISIONS:

4. Another example of *Design Decision 4* is the use of the terms partWhole and isPartOf. isPartOf is carried over from IES4 for familiarity reasons to be the partWhole relation users will commonly use.

6. isStateOf is replaced by with the more explicit isTemporalPartOf term for IES-Top.

7. To improve queries, IES-Next will limit the number of sub-properties of partWhole. E.g. replacing commonly used properties like inLocation, inPeriod, isParticipantIn with just isPartOf and isParticipationOf with isTemporalPartOf. This prevents the need to have to query for all these types of partWhole to get all parts of an extent. These changes are demonstrated in [Appendix 2](#), where we provide side-by-side comparisons of common patterns in IES4 verses how they look in IES-Next.

8. Because BoundingStates in IES4 are time intervals the set of bindings can collapse into many number of things. Consider a Person - Birth, Conception, Childhood are obvious Start bindings of persons. But the same would go for Adulthood and even Person itself - they are starting bindings of something. The same goes for finish/end bindings. Persons can die during childhood or even during birth. To avoid this issue of collapse we have opted to not carry over the partWhole relations of isStartOf and isEndOf from IES4. These relations could be interpreted as stating that this is one and only one start/end bounding. Instead, we provide the clearer relations of isAStartOf and isAFinishOf. This makes explicit that any instances of these relations are just one of many starts or ends.



Tuples and couples (1 of 2)

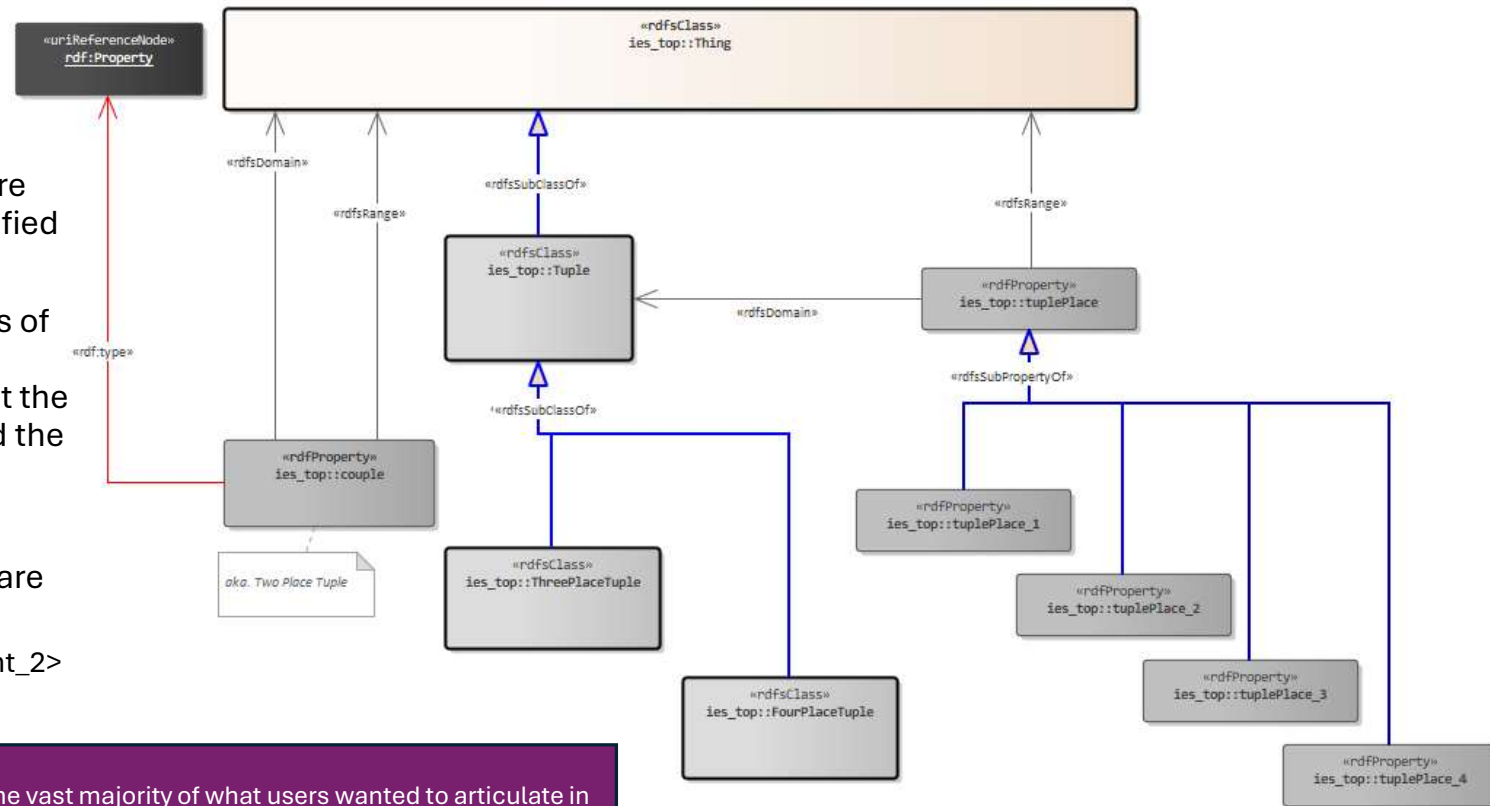
A tuple is a sequence of two or more things. Each part of a tuple is identified by a *tuplePlace*.

E.g. for the tuples that are members of *Father-Son Tuples*, you recover the father-son relations by knowing that the first tuple place is for the father and the second for the son:

<father_1, son_1>

Another example is the tuples that are members of the *Between Tuples*:

<endpoint_1, midpoint_x, endpoint_2>



DESIGN DECISIONS:

9. For IES4, we avoided higher arity Tuples as the vast majority of what users wanted to articulate in data are two-placed tuples aka. Couples. Couples were realized using simple RDF properties and this will be the same in IES-Next. However, in IES-Next we want to have a solid and complete top-level and that means having tuples that are beyond 2 places. As a result, we will support 2-placed tuples using the user-friendly RDF properties and beyond-2 placed tuples using the RDF N-ary approach which we explain in [appendix 1](#).

Tuples and couples (2 of 2)

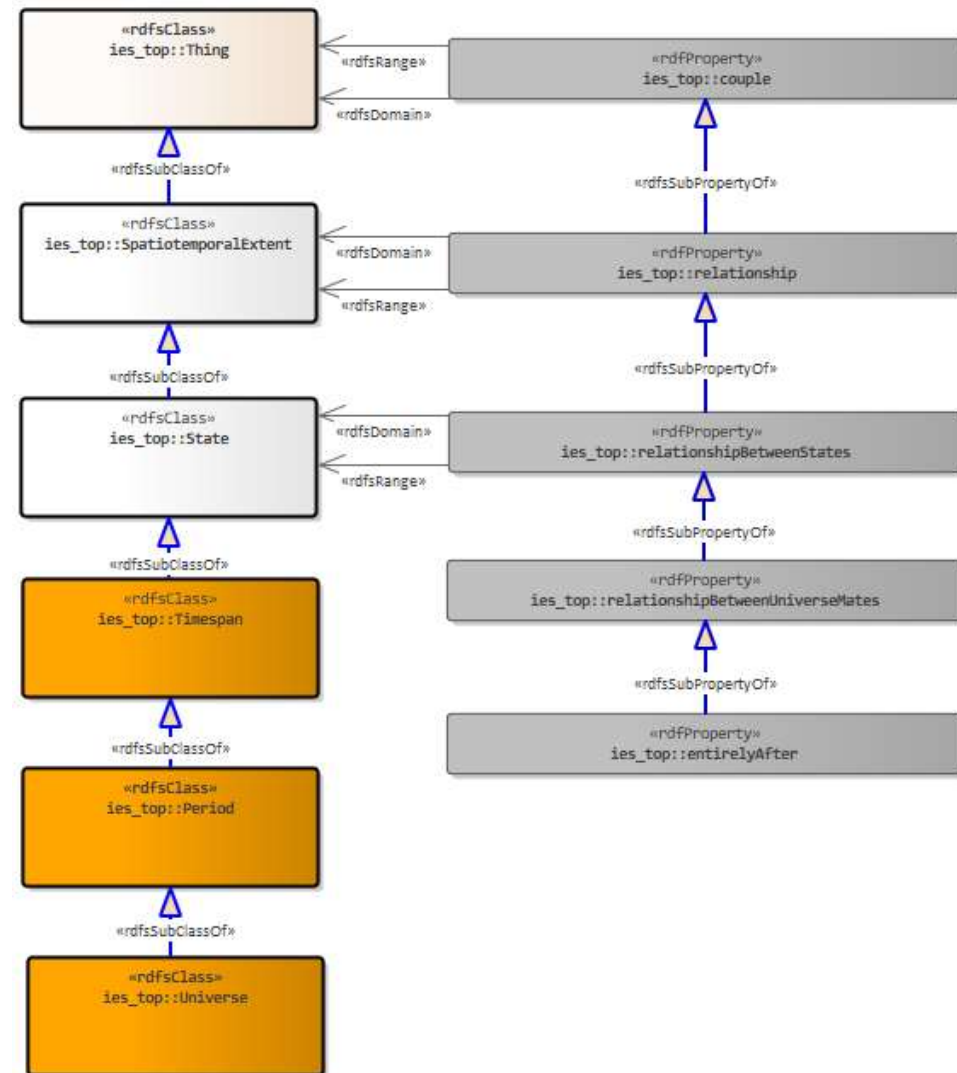
Like the mereological relations, we again make clear distinctions between relations which are between spatiotemporal extents that are universe-bound and which are not. Moreover, which relationships are between spatiotemporal extents in the same universe (universe-mates).

- **couple** – a two placed tuple. Realised in RDF as a rdf:property.
- **relationship** – a couple between any two spatiotemporal extents.
- **relationshipBetweenStates** – a relationship between any two states, where a state is a universe-bound element.
- **relationshipBetweenUniverseMates** – a relationship where those states are part of the same universe as one another.
- **entirelyAfter** – a relationshipBetweenUniverseMates where one finishes before the other starts.

Most user-created relationships that are not *grounding relations* will typically be either **ies_top:relationshipBetweenUniverseMates** or **couples**. *Couples* are utilized for relationships between spatiotemporal extents and sets, or between two sets.

DESIGN DECISIONS:

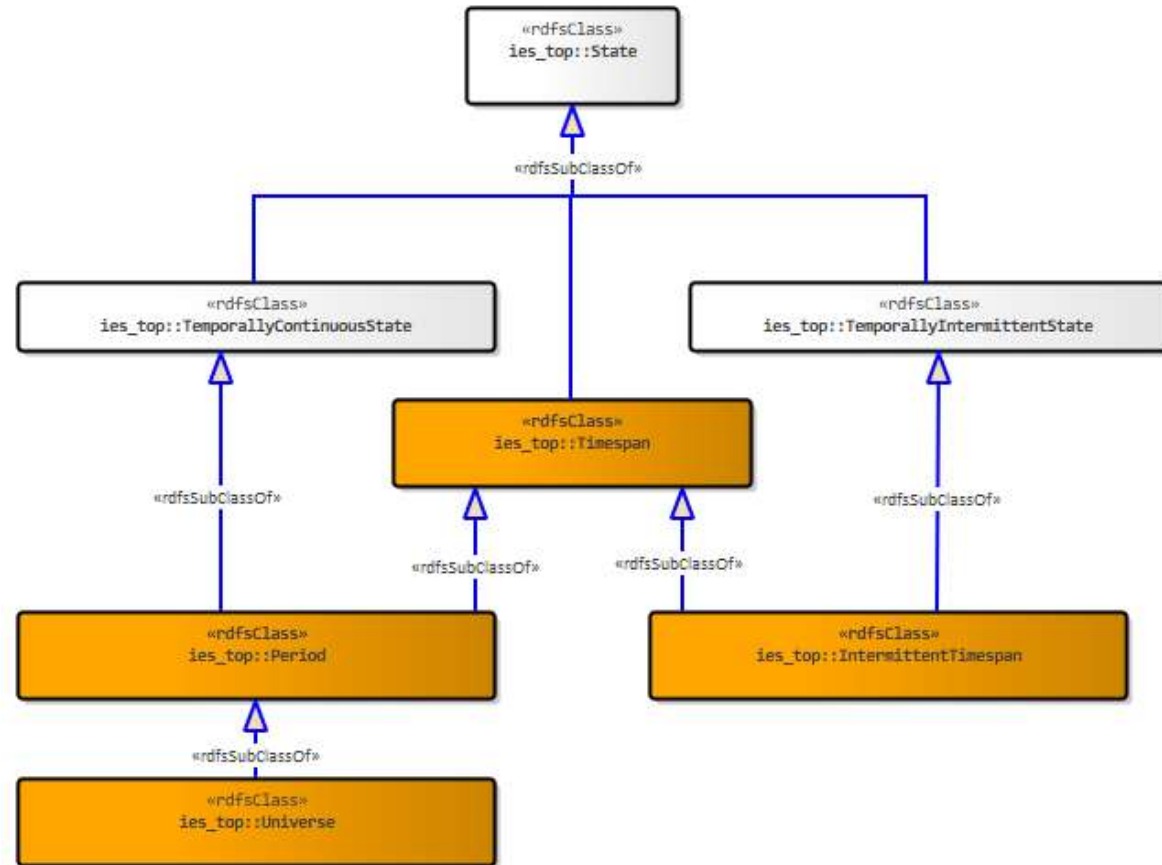
4. Another example of *Design Decision 4* is the naming of relationshipBetweenStates and relationshipBetweenUniverseMates. A balance again has been struck between being clear with what they refer to and user-friendly naming. Admittedly, the naming of these relationships has been somewhat harder than the previous examples.



Continuous & Intermittent

In IES4, we could distinguish between states which are temporally continuous i.e., without gaps in their temporal extents and those which are *gappy*. However, their definition was somewhat lost or overlooked because of their low placement in the hierarchy. In IES-Top we bring these rightly to the ...*Top*.

- **TemporallyContinuousState** – a state that is temporally continuous or temporally uninterrupted.
- **TemporallyIntermittentState** – a state that is temporally dissected - i.e. it is not continuous, but in fact a fusion of states.
- **Timespan** – a state (i.e. universe-bound element) that is a temporal part of a universe. Note: a world is an improper temporal part of itself – and so a maximal period.
- **Period** - a connected and uninterrupted timespan.
- **IntermittentTimespan** – an interrupted timespan which is also a fusion of timespans.



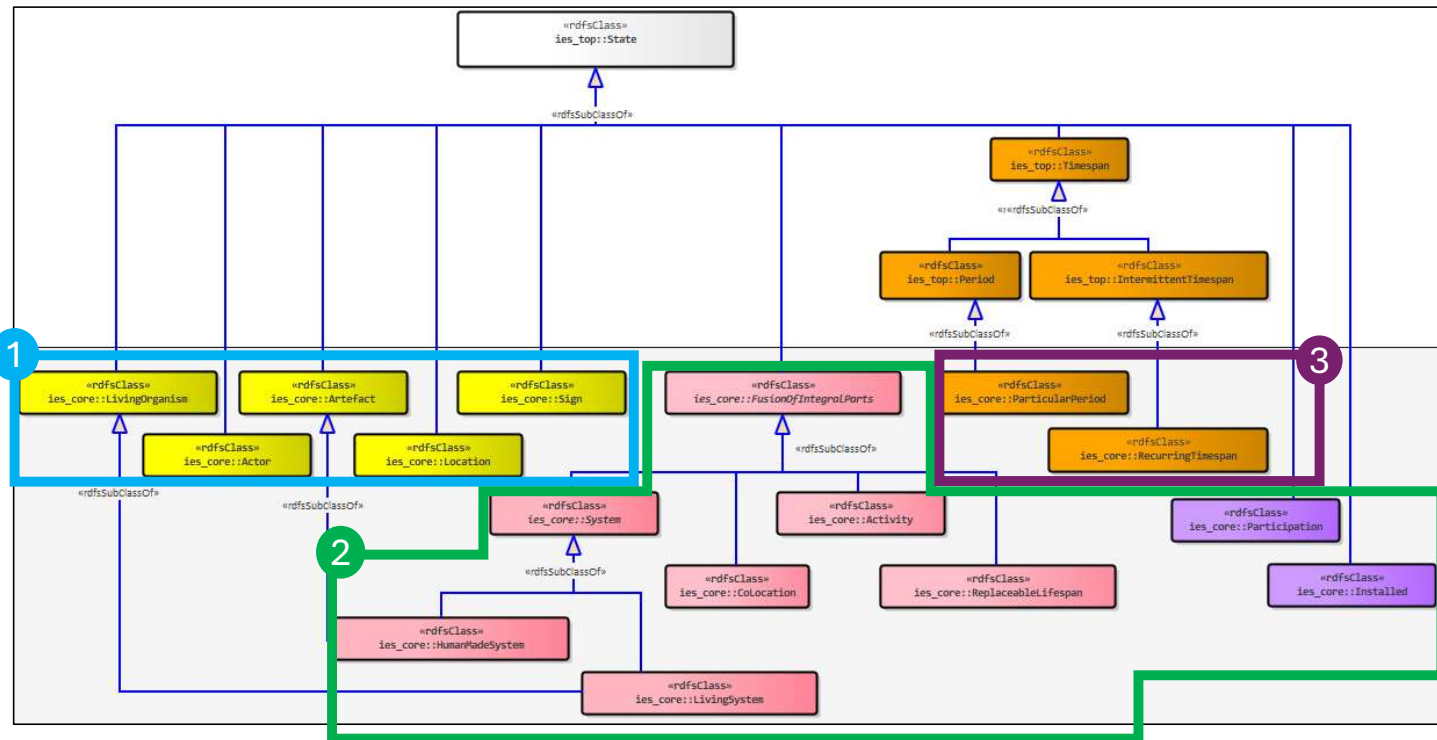
Introducing IES-Core

IES Core Overview (1 of 5)

IES-Core is built upon IES-Top providing a common foundational layer of patterns and concepts useful for expressing information in all manner of domains. Patterns such as Activities, Systems, Characteristics and Representations. This also includes lower-level concepts like Persons, Locations etc. Core is the mid-level ontology which will be the replacement of the top layers of IES4. Domain specific ontologies are expected to be built upon IES-Core.

IES-Core extends IES-Top into 4 major ways.

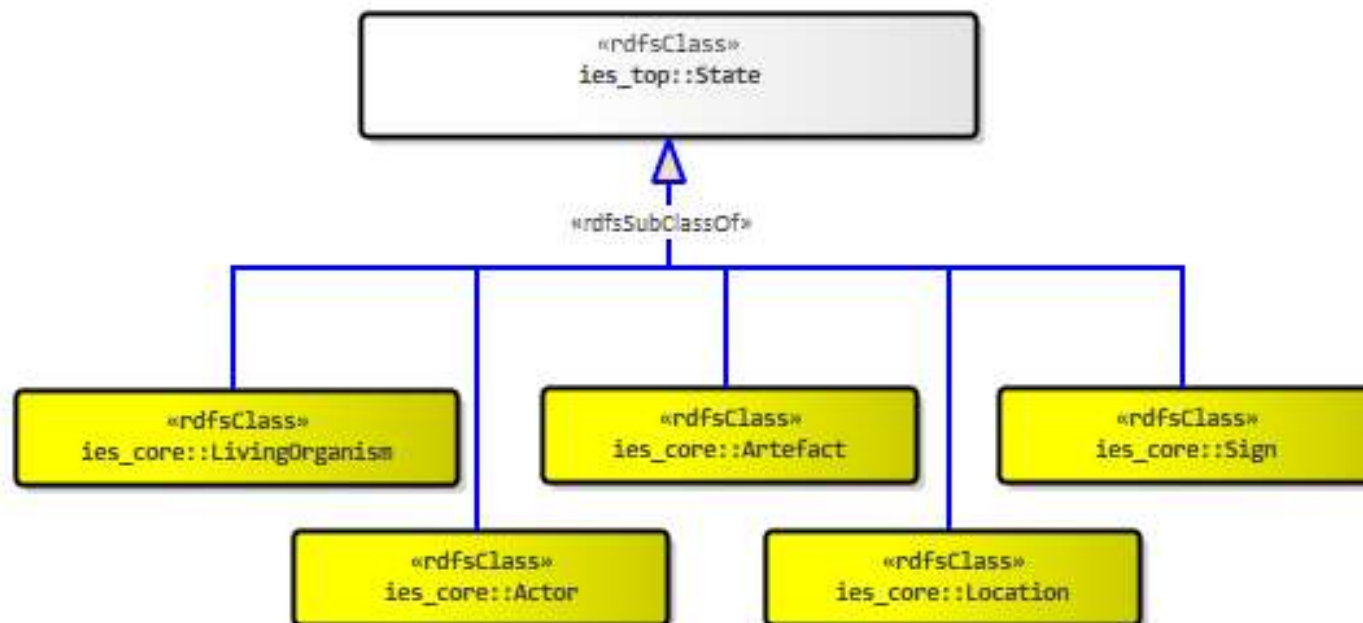
1. Extends *State* to allow us to talk about familiar tangible individuals - known as *Entities* in IES4.
2. Extends *State* to provide mechanisms to describe how states combine (fuse) to form new states of interest. Activity (Event in IES4) and System are examples of such.
3. Extends *Timespans* to represent timestamps, periods, durations, and recurrences in a 4D-faithful manner, while ensuring interoperability with common computer standards such as ISO 8601.
4. Extends *Set* to provide a Representation pattern very similar to that in IES4 (not shown here).



We will go into the details of each in subsequent slides.

IES Core Overview (2 of 5)

State is extended to allow us to talk about familiar tangible individuals - known as *Entities* in IES4. These include living individuals (*LivingOrganism*), human-made objects (*Artefact*), places (*Location*), individuals capable of conscious action or can effect change on the world (*Actor*) and individuals that symbolize or signify other individuals (*Sign*). These categories can naturally overlap, for example, a person is both a *LivingOrganism* and an *Actor*. IES-Core embraces such multiple inheritance, producing a flatter, more flexible hierarchy than IES4.

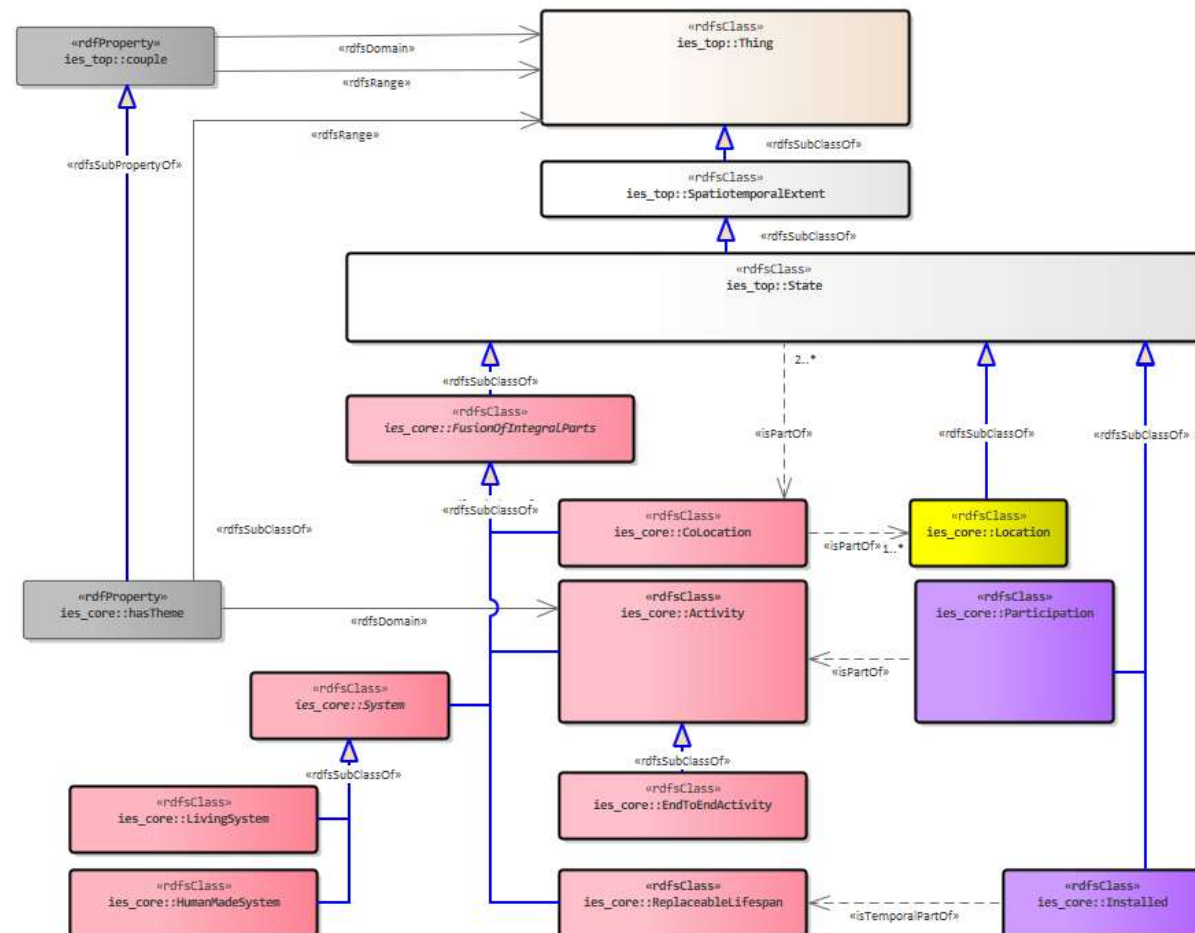


IES Core Overview (3 of 5)

State is extended to provide mechanisms to describe how states combine (fuse) to form new states of interest:

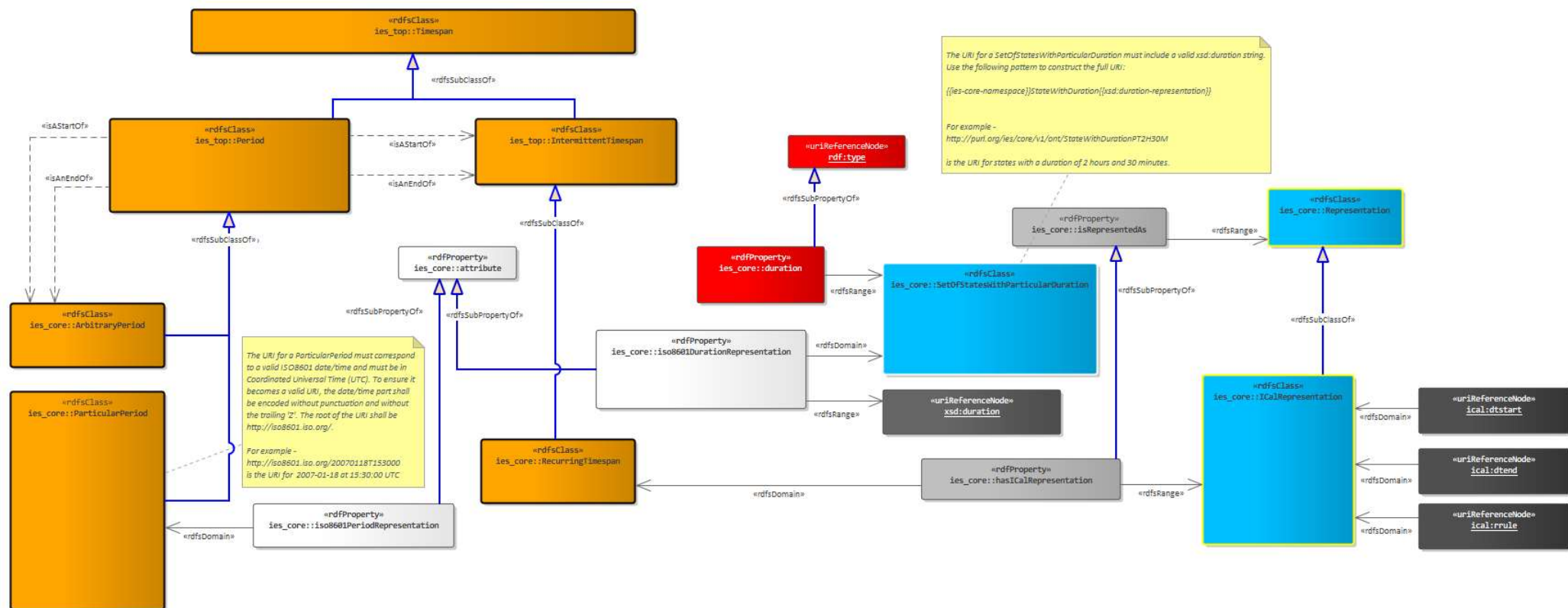
- *Activity* (Event in IES4), which is a fusion of temporal parts (*Participations*) of other states (e.g. participations of persons to make a *Meeting*);
- *System*, which is the fusion of dependent states which make up a system and which the system cannot exist without (e.g. a motorized road vehicle - made up of an engine, a chassis and wheels). Note, even seemingly tangible individuals can be treated as *Systems* - e.g. *Vehicle*, is both an *Artefact* and a *System*.

It is often useful with systems to see their parts as being *Installed* or removed but also call out the identity of dependent parts that endure. This persistence lets us refer to those parts even when they're temporarily uninstalled - for instance, the front-right wheel of a car, regardless of which actual wheel is installed at a given time. *ReplaceableLifespan* provides this timeless component but also can be used for other individuals that are replaceable outside systems e.g. the CEO of a company or the head of state of a nation.



IES Core Overview (4 of 5)

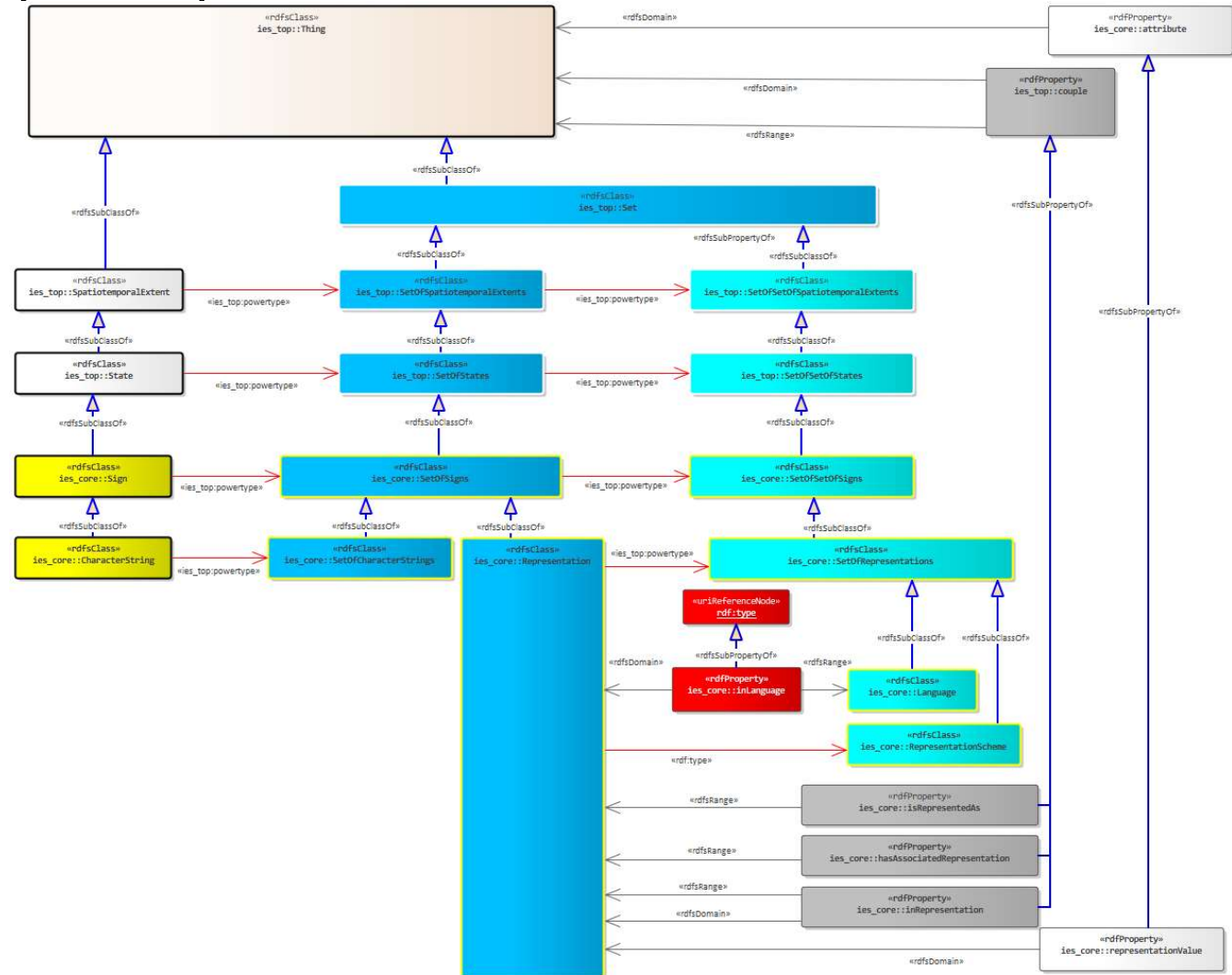
Timespan is extended to represent timestamps, periods, durations, and recurrences in a 4D-faithful manner, while ensuring interoperability with common computer standards such as ISO 8601. The structure of ParticularPeriods carries over from IES4 but in IES-Core we introduce new more interoperable ways to represent RecurringTimespan (RecurringPeriods in IES4) and durations. See [Appendix 3](#) for demonstrations of how these new mechanisms are used.



IES Core Overview (5 of 5)

Set is extended to provide a Representation pattern very similar to that in IES4.

The only major change is the previously introduced *Sign* which allows for instances of Representations (e.g. graffiti on a wall) to now be articulated in IES-Core.

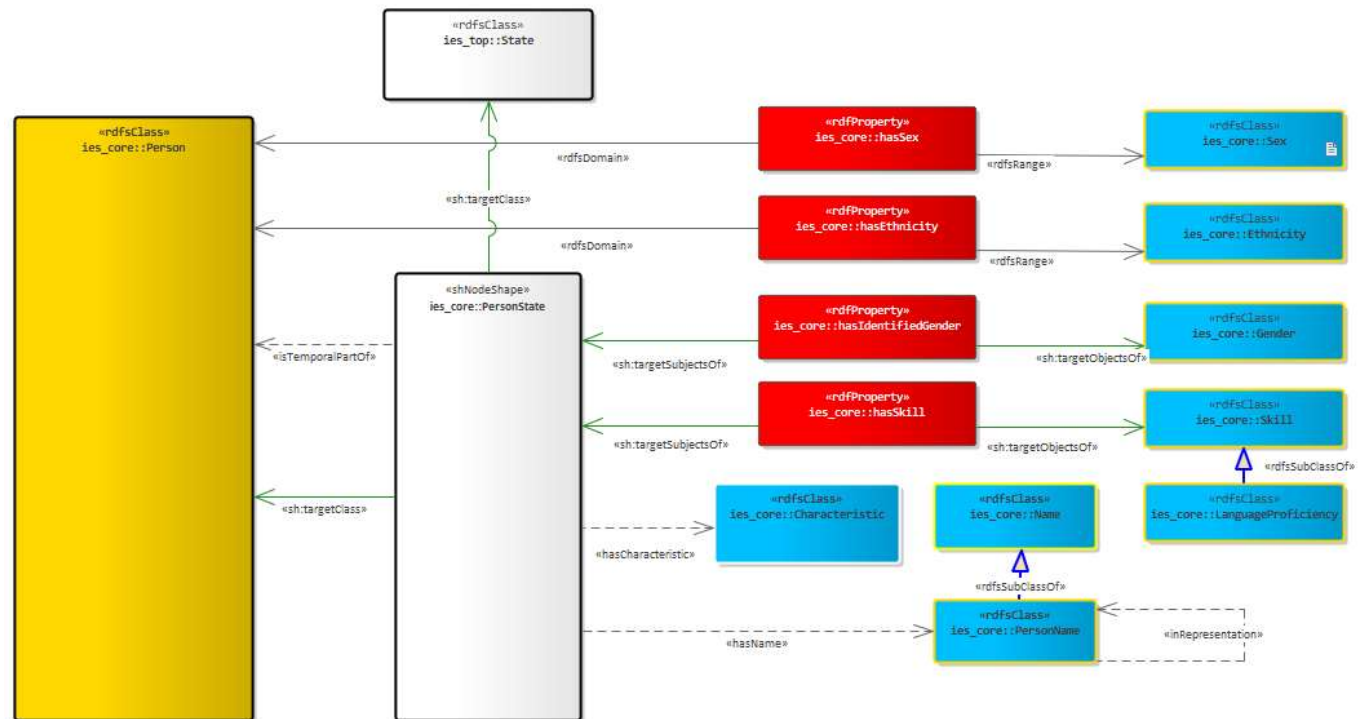


The end to IES4's Entity-State duality

In IES4 there was a mirroring hierarchy of Entities and States. E.g. Person was mirrored by PersonState, Organisation mirrored by OrganisationState. This became quite overwhelming especially when users created extensions. The original purpose of having specific states like PersonState was so that we could define the sorts of relations they can have versus others. E.g. hasIdentifiedGender can only be associated to the state of a person not other entities like organisations. Moreover, from a set-theoretic point of view, IES4's Entity collapsed into IES4's State. The IES-Core makes three changes to solve these issues:

1. Removal of Entity, ies-core goes straight to some foundational whole-life types.
2. Temporal parts of these types will be realized using ies_top:State.
3. We will use SHACL shapes to ensure the appropriate relations are made to specific types of thing. Person is a good (and the only) example of where this is needed. Note that PersonState in the diagram is not a Class in the ontology but a SHACL shape.

To see the impact of these changes in action – see [Appendix 2](#). There we provide side-by-side comparisons of common patterns in IES4 versus how they look in IES-Next



Making extensions easier

As with the previous versions of IES, IES-Core allows the model to be extended to incorporate new concepts. IES4 allowed the extension of the ontology using RDF Schema which saw users add new classes or properties using subtypes of those found in IES4. The drawback of this approach was that data consumers needed to know the specification of each custom extension or, the data producer was obliged to instantiate instances as both a class/property in IES4 and a class in their extension.

While RDF Schema extensions are still possible in IES-Core they shall only be used when the extension introduces new semantics -for example, when a new class is required to capture relationships or attributes not in IES-Core or IES-Top. Historically, most IES extensions have been purely *taxonomic* - adding narrower categories of existing concepts (e.g., more specific types of Vehicle) without introducing additional semantics.

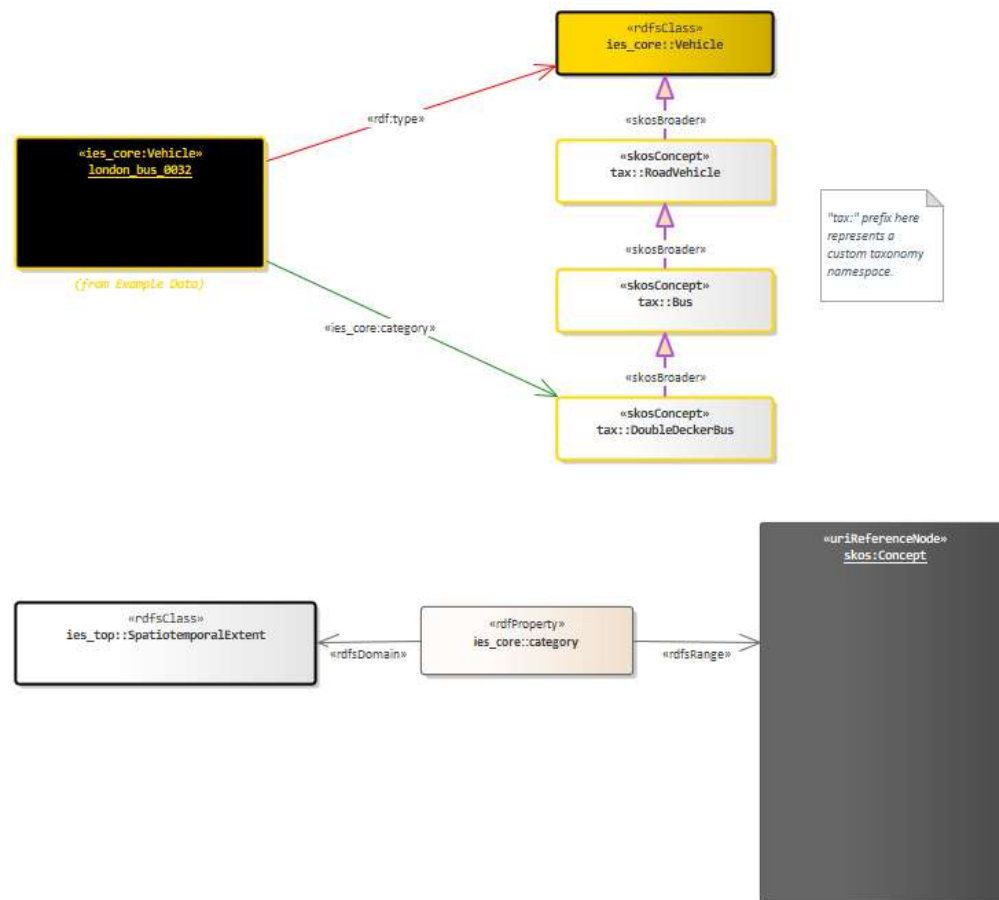
To better support this common use case, IES-Core now adopts **SKOS** (Simple Knowledge Organization System) -a lightweight RDF-based standard for controlled vocabularies such as glossaries and taxonomies.

SKOS will be used for extending IES in the following way:

1. Taxonomic extensions to IES are to be defined as instances of `skos:Concept`.
2. The `skos:broader` property is to be used to make a link from a `skos:Concept` to its parent IES class(es) (or another `skos:Concept`). `skos:narrower` shall not be used for links to IES classes - `skos:broader` already implies the inverse relationship and avoids extra handling for consumers.
3. Instances remain typed using `rdf:type`, whose range must always be an IES class - even when an extension concept is associated.
4. The `ies:category` - a property in the IES-Core ontology, shall be then used to link an instance of an IES class to a `skos:Concept`. This is what further defines it (taxonomically) as a new extension.

This regime minimizes the burden on data consumers - regardless of any custom SKOS extensions, the `rdf:type` of every instance always resolves to an IES class that all consumers understand.

In the example, an instance of a Vehicle (its `rdf:type`) is categorized as a DoubleDeckerBus (its `ies_core:category`).



BoundingState collapses into State

In IES4 and in IES-Core, BoundingStates are not instants/moments, they are temporal intervals.

Because boundings (states) are intervals and therefore contiguous in length, the set of boundings collapses into State.

Consider a person – Birth and Childhood are obvious start boundings of persons. In fact, a full-life person could be argued as an improper temporal interval related to both the start and the end of a person. Figure 1 demonstrates how Fred's Birth, Fred's Childhood and Fred are starting boundings of Fred

The same goes for end boundings. Persons can die during childhood (e.g. [Anne Frank](#)) or even during birth. Figure 2 has two examples that demonstrates the plurality of end boundings. [Sophie Scholl](#) died in adulthood so her death states, her state of being an adult and Sophie Scholl are all end boundings of Sophie Scholl. Going back to Anne Frank, because she died in childhood, the state of her being in childhood is an ending bounding of Anne Frank alongside her death state and her full-life improper part.

As a result of this collapse, in IES-Core, the temporal part which is either the start or end of a state is itself an instance of *State*. Special forms of start and end states remain e.g., BirthState and Arrival. To see the impact of the changes to starts and ends, see the first example in [Appendix 2](#).

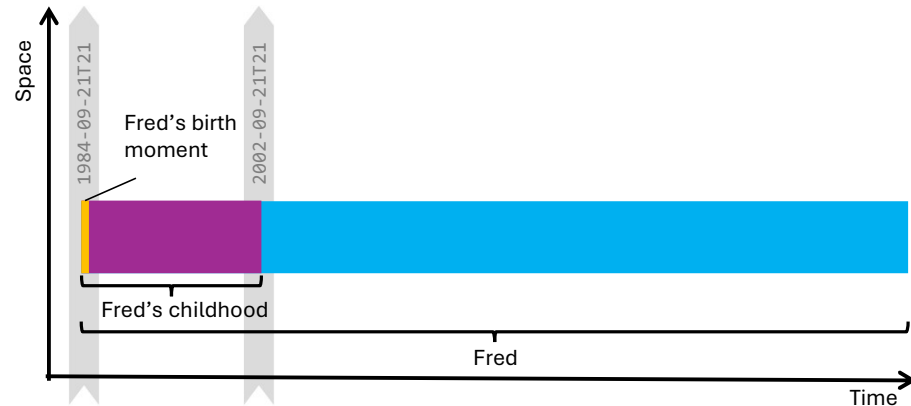


Figure 1 – Start boundings

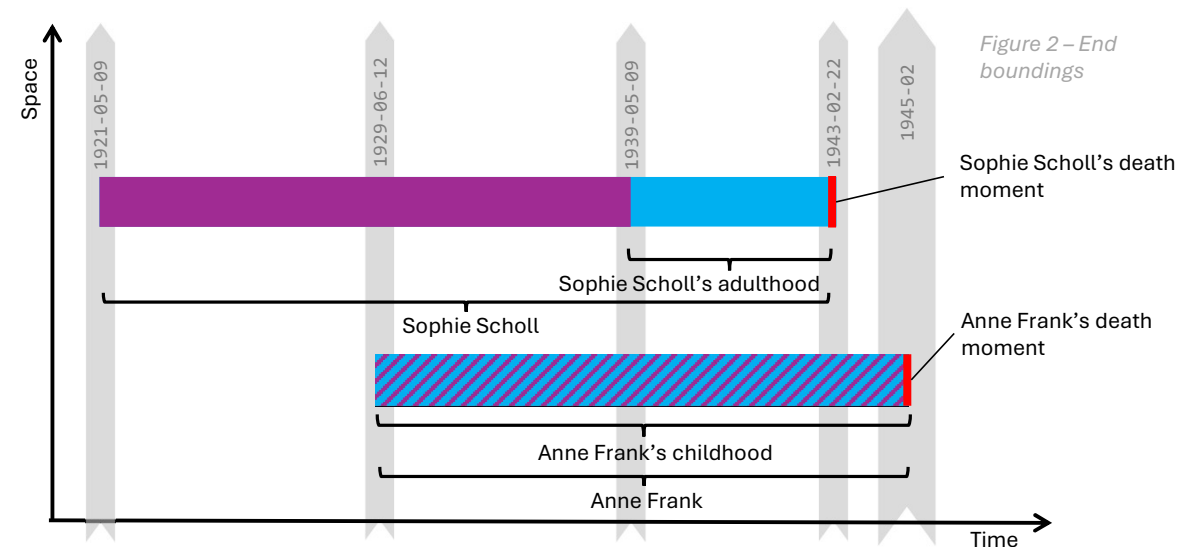
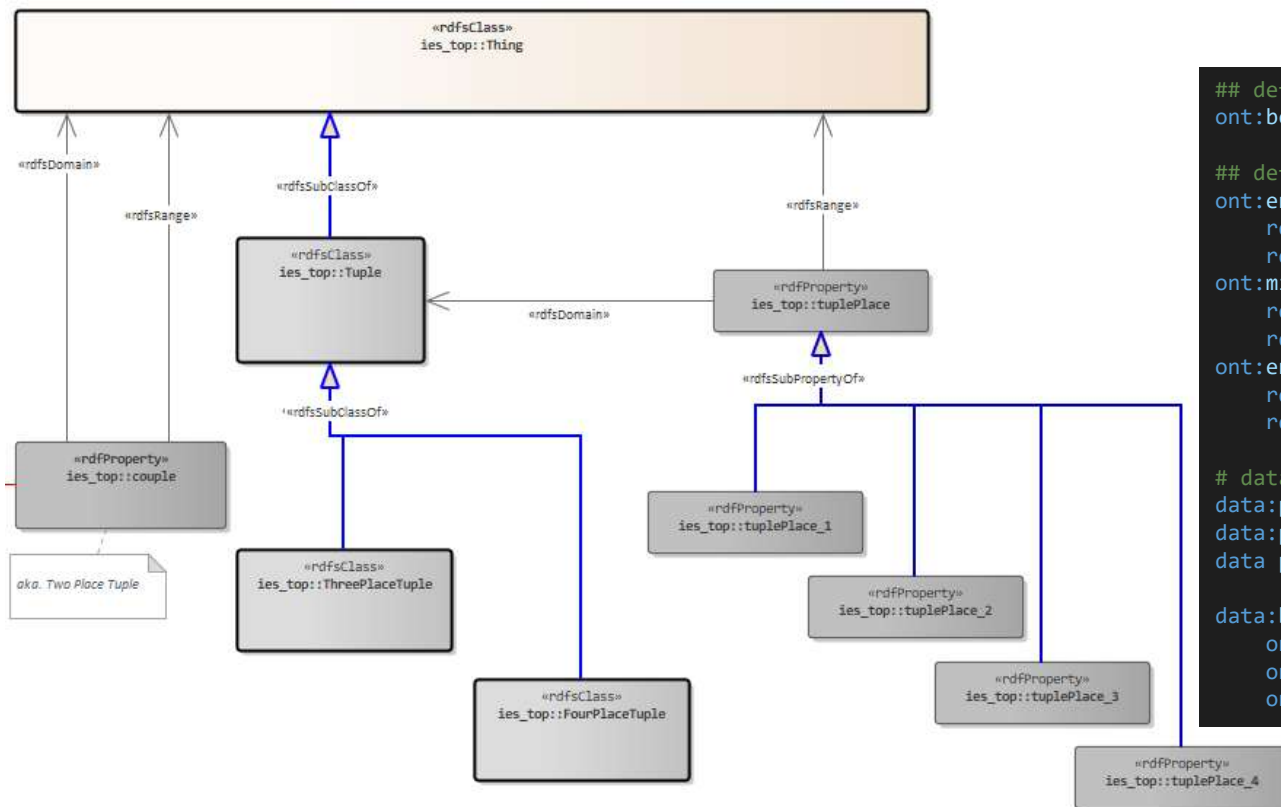


Figure 2 – End boundings

Appendix 1: Using tuples

Here we demonstrate how to specify a three-placed tuple (*Between*) in IES (using the N-ary RDF approach). We also demonstrate the use of this newly specified tuple to describe the relationship of a person between 2 other persons in a queue/line.



```

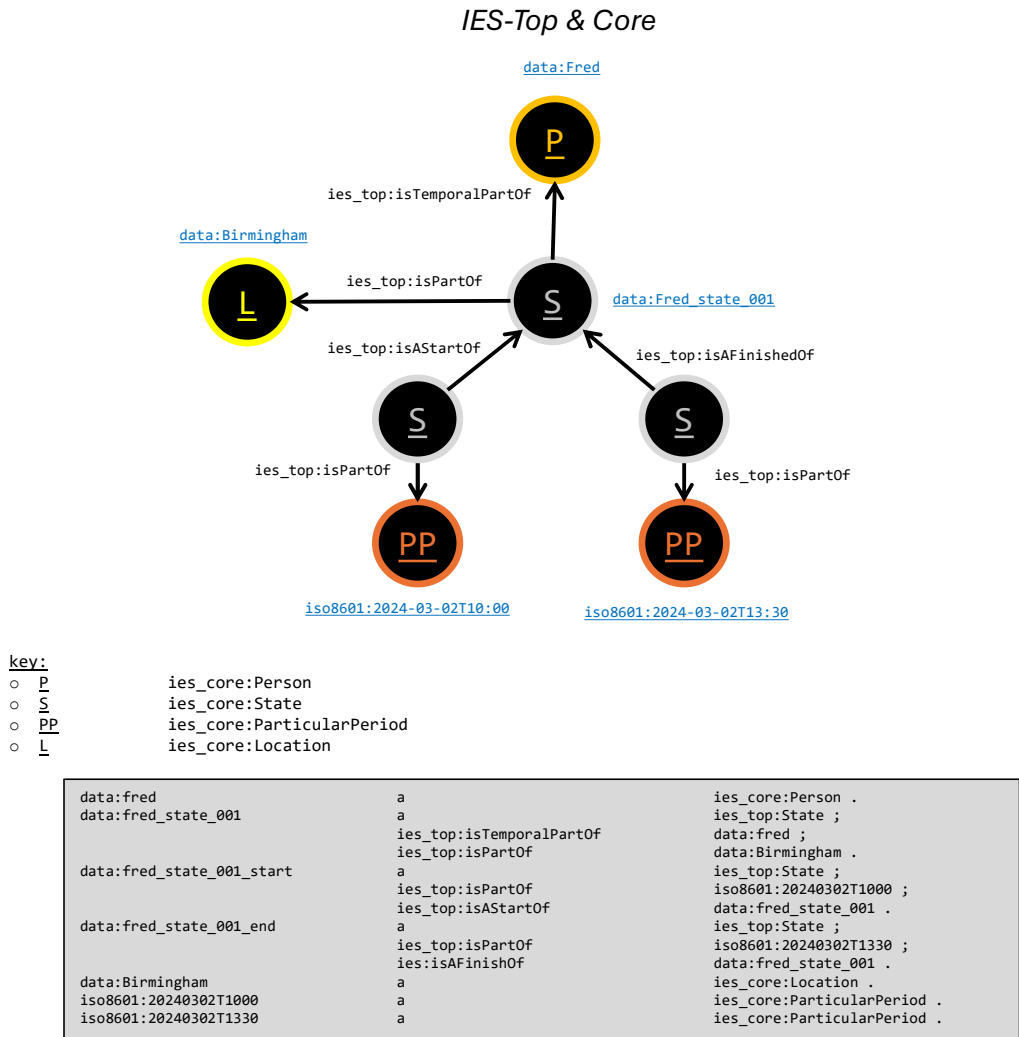
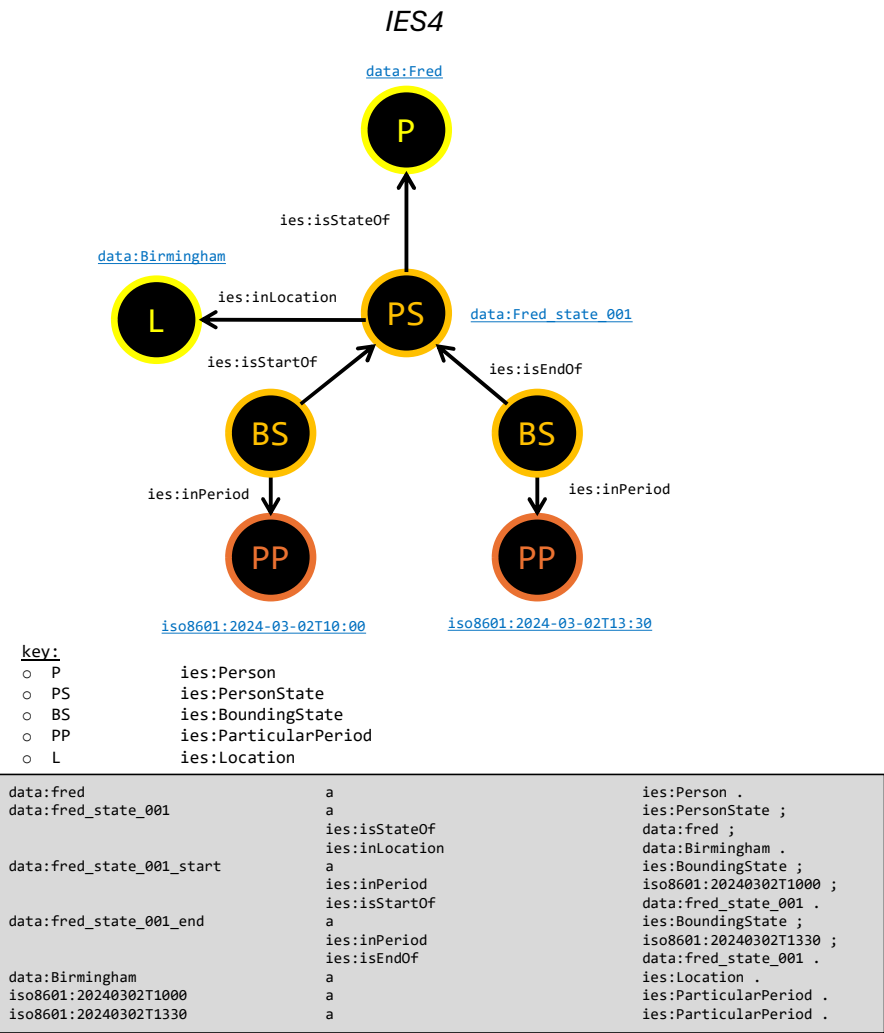
## define a new three place tuple
ont:between rdfs:subClassOf ies_top:ThreePlaceTuple .

## define the place types and their domain and range
ont:endPoint1 rdfs:subPropertyOf ies_top:tuplePlace_1 ;
  rdfs:domain ont:between ;
  rdfs:range ies_core:Person .
ont:midpoint rdfs:subPropertyOf ies_top:tuplePlace_2 ;
  rdfs:domain ont:between ;
  rdfs:range ies_core:Person .
ont:endPoint2 rdfs:subPropertyOf ies_top:tuplePlace_3 ;
  rdfs:domain ont:between ;
  rdfs:range ies_core:Person .

# data that utilised newly defined between tuple type:
data:person_1 a ies_core:Person .
data:person_2 a ies_core:Person .
data:person_3 a ies_core:Person .

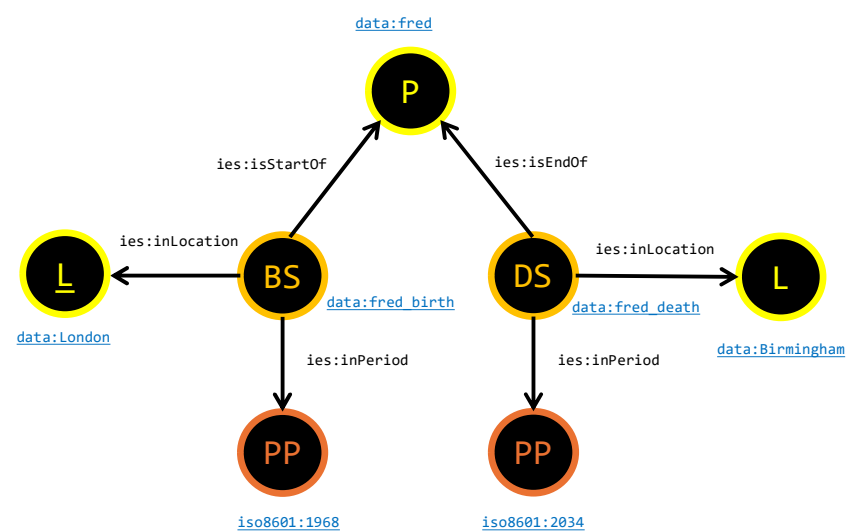
data:between_2 a ont:between ;
  ont:endPoint1 data:person_1 ;
  ont:midpoint data:person_2 ;
  ont:endPoint2 data:person_3 .
  
```


Appendix 2: IES4 vs. IES-Next comparison (1 of 3): Person’s location at some time during their life



Appendix 2: IES4 vs. IES-Next comparison (2 of 3): Person Birth and Death

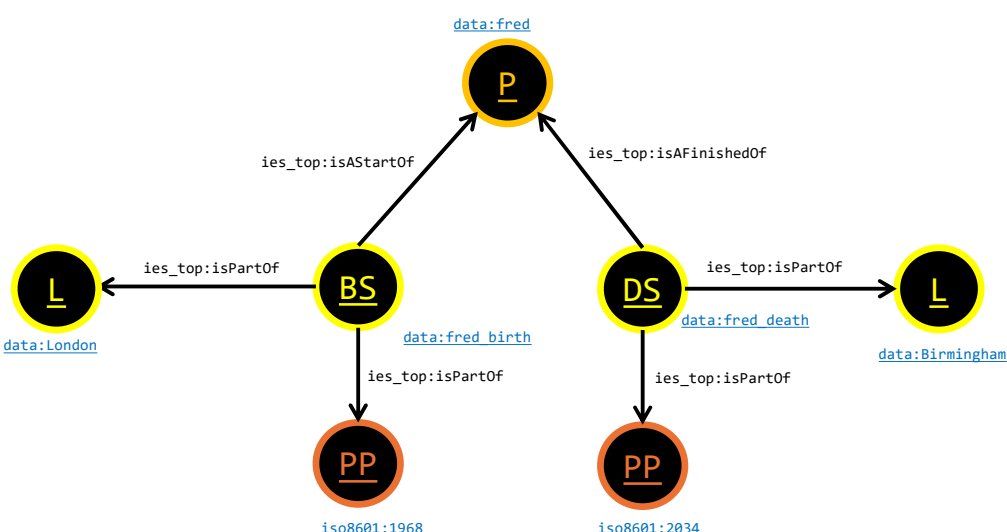
IES4



key:
o P ies:Person
o PS ies:BirthState
o DS ies:DeathState
o PP ies:ParticularPeriod
o PL ies:Location

data:fred	a	ies:Person .
data:fred_birth	a	ies:BirthState ;
	ies:isStartOf	data:fred ;
	ies:inLocation	data:London;
	ies:inPeriod	iso8601:1968 .
data:London	a	ies:Location .
iso8601:1968	a	ies:ParticularPeriod .
data:fred_death	a	ies:DeathState ;
	ies:isEndOf	data:fred ;
	ies:inLocation	data:Birmingham ;
	ies:inPeriod	iso8601:2034 .
data:Birmingham	a	ies:Location .
iso8601:2034	a	ies:ParticularPeriod .

IES-Top & Core

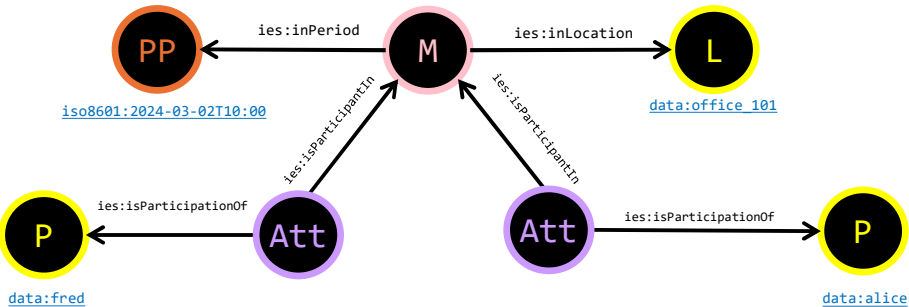


key:
o P ies_core:Person
o BS ies_core:BirthState
o DS ies_core:DeathState
o PP ies_core:ParticularPeriod
o L ies_core:Location

data:fred	a	ies_core:Person .
data:fred_birth	a	ies_core:BirthState ;
	ies_top:isAStartOf	data:fred ;
	ies_top:isPartOf	data:London;
	ies_top:isPartOf	iso8601:1968 .
data:London	a	ies_core:Location .
iso8601:1968	a	ies_core:ParticularPeriod .
data:fred_death	a	ies_core:DeathState ;
	ies_core:isAFinishOf	data:fred ;
	ies_top:isPartOf	data:Birmingham ;
	ies_top:isPartOf	iso8601:2034 .
data:Birmingham	a	ies_core:Location .
iso8601:2034	a	ies_core:ParticularPeriod .

Appendix 2: IES4 vs. IES-Next comparison (3 of 3): Meeting

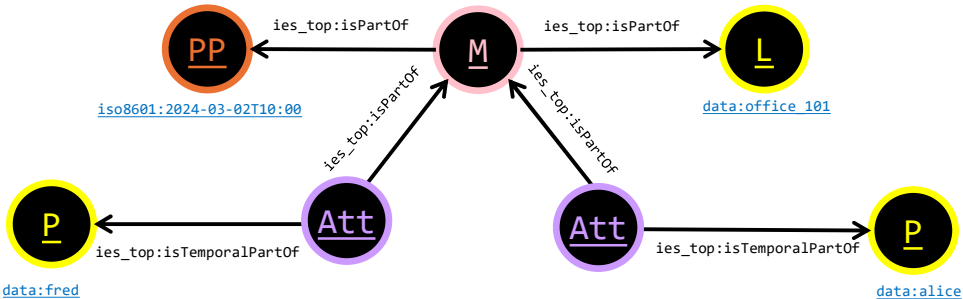
IES4



key:
o P ies:Meeting
o P ies:Person
o Att ies:Attendance
o L ies:Location
o PP ies:ParticularPeriod

data:fred	a	ies:Person .
data:alice	a	ies:Person .
data:meeting_001	a	ies:Meeting ;
	ies:inLocation	data:office_101;
	ies:inPeriod	iso8601:2024-03-02T10:00 .
data:office_101	a	ies:Location.
iso8601:2024-03-02T10:00	a	ies:ParticularPeriod .
data:fred_attendance_off_meeting_001	a	ies:Attendance ;
	ies:isParticipationOf	data:fred ;
	ies:isParticipantIn	data:meeting_001 .
data:alice_attendance_of_meeting_001	a	ies:Attendance ;
	ies:isParticipationOf	data:alice ;
	ies:isParticipantIn	data:meeting_001 .

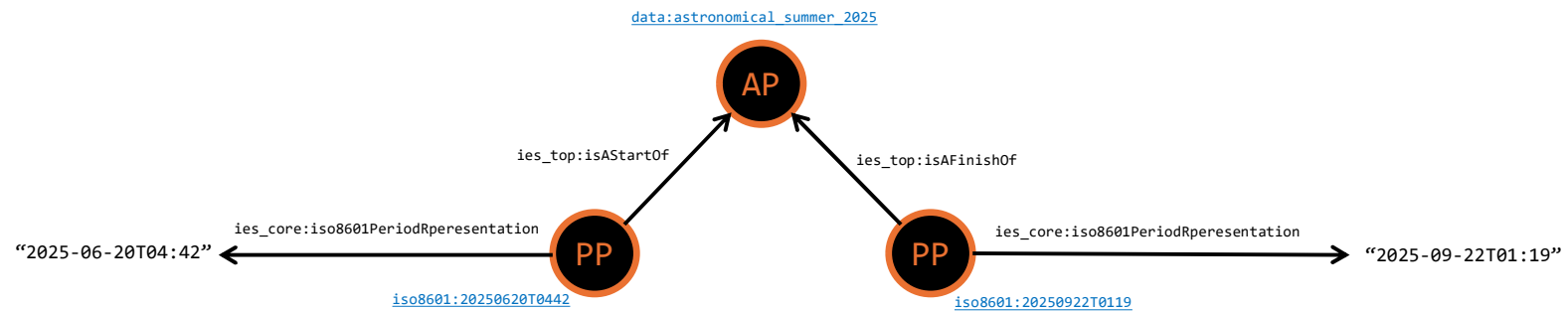
IES-Top & Core



key:
o P ies_core:Person
o M ies_core:Meeting
o Att ies_core:Attendance
o S ies_top:State
o L ies_core:Location
o PP ies_core:ParticularPeriod

data:fred	a	ies_core:Person .
data:alice	a	ies_core:Person .
data:meeting_001	a	ies_core:Meeting ;
	ies_top:isPartOf	data:office_101;
	ies_top:isPartOf	iso8601:2024-03-02T10:00 .
data:office_101	a	ies_core:Location.
iso8601:2024-03-02T10:00	a	ies_core:ParticularPeriod .
data:fred_attendance_of_meeting_001	a	ies_core:Attendance ;
	ies_top:isTemporalPartOf	data:fred ;
	ies_top:isPartOf	data:meeting_001 .
data:alice_attendance_of_meeting_001	a	ies_core:Attendance ;
	ies_top:isTemporalPartOf	data:alice ;
	ies_top:isPartOf	data:meeting_001 .

Appendix 3 (1 of 3): ParticularPeriod and ArbitraryPeriod



key:

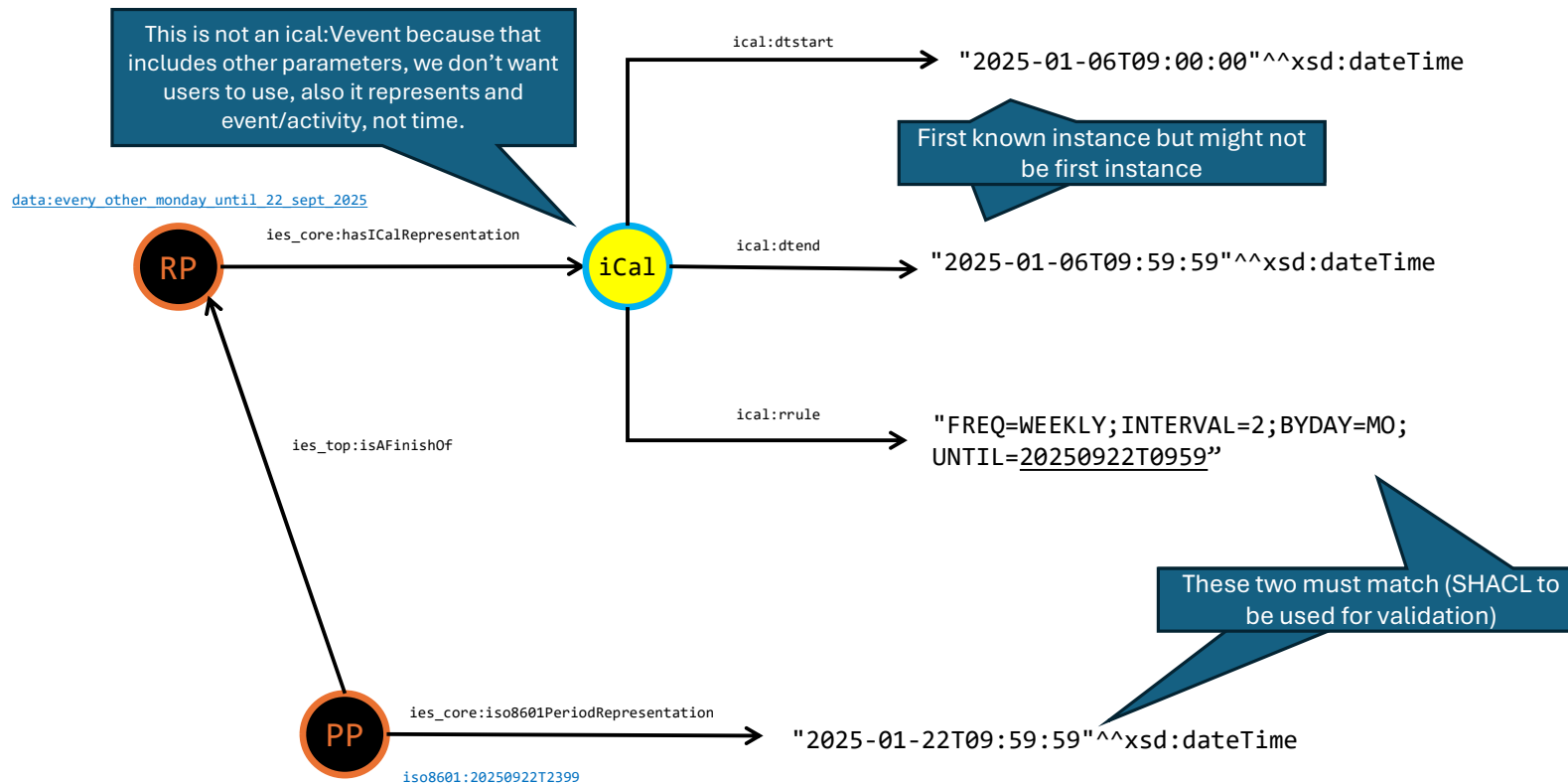
- P ies_core:ParticularPeriod
- AP ies_core:ArbitraryPeriod

Appendix 3 (2 of 3): RecurringPeriod

In this example, I know the recurrence pattern, know when the recurrences ended but do not know when they began.

key:

- P ies_core:ParticularPeriod
- RP ies_core:RecurringPeriod
- iCal ies_core:ICalRepresentation



Appendix 3 (3 of 3): Duration

Example of a meeting that took 2 hours and 30 mins.

